KISS-ICP: In Defense of Point-to-Point ICP – Simple, Accurate, and Robust Registration If Done the Right Way

Ignacio Vizzo Tiziano Guadagnino Benedikt Mersch Louis Wiesmann Jens Behley Cyrill Stachniss

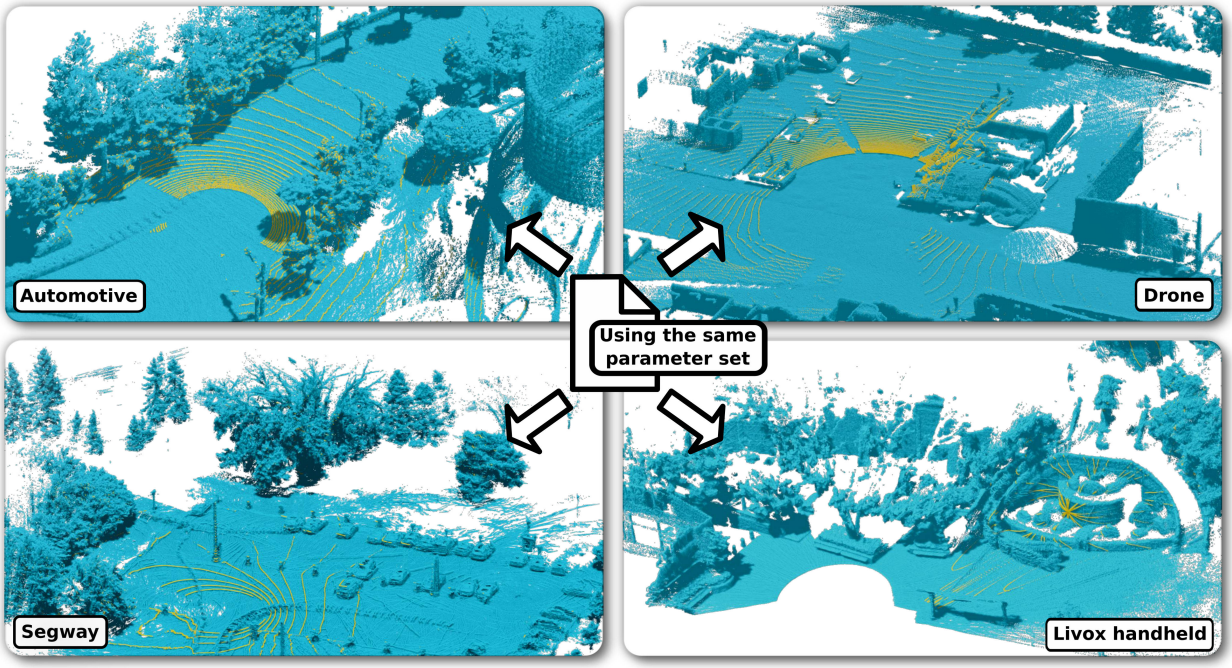


Fig. 1: Point cloud maps (blue) generated by our proposed odometry pipeline on different datasets with the same set of parameters. We depict the latest scan in yellow. The scans are recorded using different sensors with different point densities, different orientations, and different shooting patterns. The automotive example stems from the MulRan dataset [15]. The drone of the Voxgraph dataset [23] and the segway robot used in the NCLT dataset [5] show a high acceleration motion profile. The handheld Livox LiDAR [17] has a completely different shooting pattern than the commonly used rotating mechanical LiDAR.

***Abstract*—Robust and accurate pose estimation of a robotic platform, so-called sensor-based odometry, is an essential part of many robotic applications. While many sensor odometry systems made progress by adding more complexity to the ego-motion estimation process, we move in the opposite direction. By removing a majority of parts and focusing on the core elements, we obtain a surprisingly effective system that is simple to realize and can operate under various environmental conditions using different LiDAR sensors. Our odometry estimation approach relies on point-to-point ICP combined with adaptive thresholding for correspondence matching, a robust kernel, a simple but widely applicable motion compensation approach, and a point cloud subsampling strategy. This yields a system with only a few parameters that in most cases do not even have to be tuned to a specific LiDAR sensor. Our system performs on par with state-of-the-art methods under various operating conditions using different platforms using the same parameters: automotive platforms, UAV-based operation, vehicles like segways, or handheld LiDARs. We do not require integrating IMU data and solely rely on 3D point clouds obtained from a wide range of 3D LiDAR sensors, thus, enabling a broad spectrum of different applications and operating conditions. Our open-source system operates faster than the sensor frame rate in all presented datasets and is designed for real-world scenarios.**

***Index Terms*—Mapping; Localization; SLAM**

I. Introduction

ODOMETRY estimation is an essential building block for any mobile robot that needs to autonomously navigate in unknown environments. In the LiDAR sensing domain, current odometry pipelines typically use some form of iterative closest point (ICP) to estimate poses incrementally [10], [26], [31], [35]. Even though LiDAR odometry has been an active area of research for the last three decades, the design of current systems is usually coupled with assumptions about the robot motion [10] and the structure of the environment [28] to

Manuscript received: September 14, 2022; Revised: December 5, 2022; Accepted: December 27, 2022. This paper was recommended for publication by Editor Javier Civera upon evaluation of the Associate Editor and Reviewers' comments.

This work has partially been funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy, EXC-2070 – 390732324 – PhenoRob and by the European Union's HORIZON research and innovation programme under grant agreement No 101070405 (Digiforest).

All authors are with the University of Bonn, Germany. Cyrill Stachniss is additionally with the Department of Engineering Science at the University of Oxford, UK, and with the Lamarr Institute for Machine Learning and Artificial Intelligence, Germany.

Digital Object Identifier (DOI): see top of this page.

achieve accurate and robust alignment results. To the best of our knowledge, no existing 3D LiDAR odometry approach is free of parameter tuning and works out of the box in different scenarios, using arbitrary LiDAR sensors, supporting different motion profiles, and consequently types of robots, such as ground and aerial robots.

This paper returns to the roots: classical point-to-point ICP, introduced 30 years ago by Besl and McKay [3]. We aim to tackle the inherent problems of sequentially operating LiDAR odometry systems that prohibit current approaches from generalizing to different environments, sensor resolutions, and motion profiles using a single configuration. We present simple yet effective reasoning about the robot kinematics and the sequential way LiDAR data is recorded on a mobile platform, as well as an effective downsampled point cloud representation that allows us to minimize the need for parameter tuning.

Our system challenges even extensively hand-tuned and optimized existing simultaneous localization and mapping (SLAM) systems. Our design uses neither sophisticated feature extraction techniques, learning methods, nor loop closures. The same parameter set works in various challenging scenarios such as highway drives of robot cars with many dynamic objects, drone flights, handheld devices, segways, and more. Thus, we take a step back from mainstream research in LiDAR odometry estimation and focus on reducing the components to their essentials. This makes our system perform extraordinarily well in various real-world scenarios, see Fig. 1.

The main contribution of this paper is a simple yet highly effective approach for building LiDAR odometry systems that can accurately compute a robot's pose online while navigating through an environment. We identify the core components and properly evaluate the impact of different modules on such systems. We show that with the proper use of ICP that builds on basic reasoning about the system's physics and the sensor data's nature, we obtain competitive odometry. Besides motion prediction, spatial scan downsampling, and a robust kernel, we introduce an adaptive threshold approach for ICP in the context of robot motion estimation that makes our approach effective and, at the same time, generalizes easily.

We make three key claims: Our "keep it small and simple" approach exploiting point-to-point ICP is (i) on par with state-of-the-art odometry systems, (ii) can accurately compute the robot's odometry in a large variety of environments and motion profiles with the same system configuration, and (iii) provides an effective solution to motion distortion without relying on IMUs or wheel odometers. In sum, "good old point-to-point ICP" is a surprisingly powerful tool, and there is little need to move to more sophisticated approaches if the basic components are done well.

We provide an open-source implementation at: https://github.com/PRBonn/kiss-icp that precisely follows the description of this paper.

II. Related Work

Point cloud registration has been an active area of research for the last three decades [3], [9] and is still relevant nowadays. The ICP algorithm can solve the problem of finding a transformation that brings two different point clouds into a common reference frame, and it is a special case of the absolute orientation problem in photogrammetry. ICP typically consists of two parts. The first one is to find correspondences between the point clouds. The second one computes the transformation that minimizes an objective function defined on the correspondences from the first step. One repeats this process until a convergence criterion is met. Most ICP variants [1], [10], [11], [26], [21], [35] utilize a maximum distance threshold in the data association module plus a robust kernel [6] and a maximum number of iterations. In contrast, we propose a threshold estimation method that adapts to changing scenarios by reasoning about the system kinematics and the nature of the data in combination with a robust kernel. We avoid controlling the number of iterations of the ICP to achieve better generalization.

ICP can be used to obtain an odometry estimation from streaming data from a sensor such as RGB-D cameras [19] or LiDARs [10]. In this work, we focus on the problem of LiDAR odometry estimation, although the ideas presented can be easily extended to other range-sensing technologies.

Nearly all modern SLAM systems build on top of odometry algorithms. Zhang et al. [35] proposed lidar odometry and mapping (LOAM) that computes the robot's odometry by registering planar and edge features to a sparse feature map. LOAM inspired numerous other works [27], [33], such as Lego-LOAM [28], which adds ground constraints to improve accuracy, and recently F-LOAM [33], which revised the original method with a more efficient optimization technique enabling faster operation. However, these methods rely on hand-tuned feature extraction, which typically requires tedious parameter tuning that depends on sensor resolution, environment structure, etc. In contrast, we only rely on point coordinates removing this data-dependent parameter adaptation.

Behley and Stachniss [1] propose the surfel-based method SuMa to achieve LiDAR odometry estimation and mapping. It has also been extended to account for semantics [8] and explicitly handle dynamic objects [7]. In contrast to the surfel-based mapping, Deschaud [11] introduced IMLS-SLAM [11] selecting an implicit moving least square surface [16] as map representation. Along these lines, Vizzo et al. [31] exploited a triangular mesh as the internal map representation. All the above approaches rely on a point-to-plane [24] metric to register consecutive scans. This requires normal estimation, which introduces additional data-dependent parameters. Furthermore, noisy 3D information can impact the normal computation and subsequently the registration in a negative way. We will show that by minimizing a simpler point-to-point metric, we obtain on-par or better odometry performance. Moreover, this design choice enables us to represent the internal map as a voxelized, downsampled point cloud, simplifying the implementation.

Recently, several new approaches [10], [21], [27] have been proposed to solve the odometry estimation problem. Most of these works focus on the runtime operation of the system as well as on the accuracy. Pan et al. [21] propose a multi-metric system (MULLS) that obtains good results in many challenging scenarios at the cost of tuning many parameters for each run. Dellenbach et al. [10] introduced a novel approach, called continuous time ICP (CT-ICP), which incorporates the

motion un-distortion into the registration showing great results but adding more complexity. Additionally, the robots' motion profile must be known a priori, as, for example, a car will have a different profile than a segway platform. We challenge the need for sophisticated optimization techniques to cope with motion distortion requiring only the constant velocity model. Furthermore, our system only relies on a few parameters, and we do not need to know the motion profile in advance.

Many state-of-the-art odometry systems [1], [10], [21], [27] also rely on pose graph optimization to achieve a better alignment. In contrast, we do not exploit such techniques and state that pose graph optimization is orthogonal to the presented approach and can be easily integrated. In sum, we step back from the common mainstream work on LiDAR odometry and propose a system that solely relies on a point-to-point metric and does not employ pose graph optimization [1], [10], [21], [27]. Our system can run on different types of mobile robots, drones, handheld devices, and segways, without the need to fine-tune the system to a specific application.

III. KISS-ICP – Keep It Small and Simple

This work aims to incrementally compute the trajectory of a moving LiDAR sensor by sequentially registering the point clouds recorded by the scanner. We reduced the components to a minimal set needed to build an effective, accurate, robust, and still reasonably simple LiDAR odometry system.

For each 3D scan in form of a local, egocentric point cloud $\mathcal{P} = \{\boldsymbol{p}_i \,|\, \boldsymbol{p}_i \in \mathbb{R}^3\}$, we perform the following four steps to obtain a global pose estimate $\mathsf{T}_t \in SE(3)$ at time t. First, we apply sensor motion prediction and motion compensation, often called deskewing, to undo the distortions of the 3D data caused by the sensor's motion during scanning. Second, we subsample the current scan. Third, we estimate correspondences between the input point cloud and a reference point cloud, which we call the local map. We use an adaptive thresholding scheme for correspondence estimation, restricting possible data associations and filtering out potential outliers. Fourth, we register the input point cloud to the local map using a robust point-to-point ICP algorithm. Finally, we update the local map with a downsampled version of the registered scan. Below, we describe these components in detail.

A. Step 1: Motion Prediction and Scan Deskewing

We advocate for rethinking the point cloud registration in the context of mobile robots, which continuously record data. One should not think of it as registering arbitrary pairs of 3D point clouds. Instead, one should phrase it as *estimating how much the robot's actual motion deviates from its expected motion by registering consecutive scans*.

Different approaches can be used to compute the robot's expected motion before considering the LiDAR data. The three most popular choices are the constant velocity model, wheel odometry obtained through encoders, and IMU-based motion estimation. The constant velocity [29] model assumes that a robot moves with the same translational and rotational velocity as in the previous time step. It requires no additional sensors (no wheel encoder, no IMU) and thus is the most widely applicable option.

Our approach uses the constant velocity model for two reasons: first, it is generally applicable, requires no additional sensors, and avoids the need for time synchronization between sensors. Second, as we will show in our experimental evaluation, it works well enough to provide a solid initial guess when searching for data associations and deskewing 3D scans. This follows from the fact that robotic LiDAR sensors commonly record and stream point clouds at 10 Hz to 20 Hz, i.e., every 0.05 s to 0.1 s. In most cases, the acceleration or deceleration, i.e., the deviations from the constant velocity model that occurs within such short time intervals, are fairly small. If the robot accelerates or decelerates, the constant velocity estimation of the robot's pose will be slightly off, and therefore, we need to correct this estimate through registration. These accelerations determine the possible displacements of the (static) 3D points.

The constant velocity model approximates the translational and angular velocities, denoted as $\boldsymbol{v}_t$ and $\boldsymbol{\omega}_t$ at time t respectively, by using the previous pose estimates $\mathsf{T}_{t-1} = (\boldsymbol{R}_{t-1}, \boldsymbol{t}_{t-1})$ and $\mathsf{T}_{t-2} = (\boldsymbol{R}_{t-2}, \boldsymbol{t}_{t-2})$, represented by a rotation matrix $\boldsymbol{R}_t \in SO(3)$ and a translation vector $\boldsymbol{t}_t \in \mathbb{R}^3$ for the time step t. We first compute the relative pose $\mathsf{T}_{\text{pred},t}$ that we will use as motion prediction as:

$$\mathsf{T}_{\text{pred},t} = \begin{bmatrix} \boldsymbol{R}_{t-2}^\top \boldsymbol{R}_{t-1} & \boldsymbol{R}_{t-2}^\top (\boldsymbol{t}_{t-1} - \boldsymbol{t}_{t-2}) \\ \boldsymbol{0} & 1 \end{bmatrix}, \tag{1}$$

then derive the corresponding velocities as:

$$\boldsymbol{v}_t = \frac{\boldsymbol{R}_{t-2}^\top (\boldsymbol{t}_{t-1} - \boldsymbol{t}_{t-2})}{\Delta t}, \tag{2}$$

$$\boldsymbol{\omega}_t = \frac{\operatorname{Log}(\boldsymbol{R}_{t-2}^\top \boldsymbol{R}_{t-1})}{\Delta t}, \tag{3}$$

where Δt is the acquisition time of one LiDAR sweep, typically 0.05 s or 0.1 s, and Log: $SO(3) \to \mathbb{R}^3$ extracts the axis-angle representation.

Note that also wheel odometry or an IMU-based motion prediction approach can be used instead to compute $\boldsymbol{v}_t$ and $\boldsymbol{\omega}_t$ for each time step. This will not change the remainder of our approach. For example, if one has good wheel odometry available, this can also be used. However, we use constant velocity as a generally applicable approach.

Within the acquisition time Δt of one LiDAR sweep, multiple 3D points are measured by the scanner. The relative timestamp $s_i \in [0, \Delta t]$ for each point $\boldsymbol{p}_i \in \mathcal{P}$ describes the recording time relative to the scan's first measurement. This relative timestamp allows us to compute the motion compensation resulting in a deskewed point $\boldsymbol{p}_i^* \in \mathcal{P}^*$ of the corrected scan $\mathcal{P}^*$ reading by

$$\boldsymbol{p}_i^* = \operatorname{Exp}(s_i \boldsymbol{\omega}_t) \boldsymbol{p}_i + s_i \boldsymbol{v}_t, \tag{4}$$

where Exp: $\mathbb{R}^3 \to SO(3)$ computes a rotation matrix from an axis-angle representation. Note that $\operatorname{Exp}(s_i \boldsymbol{\omega}_t)$ is equivalent to performing SLERP in the axis-angle domain.

This form of scan deskewing, especially with the constant velocity model, is easy to implement, generally applicable, and does not require additional sensors, high-precision time synchronization between sensors, or IMU biases to be estimated.

As we show in Sec. IV, this approach often performs even better than more complex compensation systems [10], at least as long the motion between the start and end of the sweep is small as it is for most robotics applications.

B. Step 2: Point Cloud Subsampling

Identifying a set of keypoints in the point cloud is a common approach for scan registration [14], [24], [35]. It is typically done to achieve faster convergence and/or higher robustness in the data association. However, complex filtering of the point cloud usually comes with an extra layer of complexity and parameters that often need to be tuned.

Rather than extracting 3D keypoints, which often requires environment-dependent parameter tuning, we propose to compute only a spatially downsampled version $\hat{\mathcal{P}}^*$ of the deskewed scan $\mathcal{P}^*$. Downsampling is done using a voxel grid. As we will explain in Sec. III-C below in more detail, we use a voxel grid as our local map, where each voxel call has a size of $v \times v \times v$ and each cell only store a certain number of points. Every time we process an incoming scan, we first downsample the point cloud of the scan to an intermediate point cloud $\mathcal{P}^*_{\text{merge}}$, which is later used to update the map when the relative motion of the robot has been determined with ICP. To obtain the points in $\mathcal{P}^*_{\text{merge}}$, we use voxel size $\alpha\, v$ with $\alpha \in (0.0, 1.0]$ and keep only a single point per voxel.

For the ICP registration, an even lower resolution scan is beneficiary. Thus, we compute a further reduced point cloud $\hat{\mathcal{P}}^*$ by downsampling $\mathcal{P}^*_{\text{merge}}$ again using a voxel size of $\beta\, v$ with $\beta \in [1.0, 2.0]$ keeping only a single point per voxel. This further reduces the number of points processed during the registration and allows for a fast and highly effective alignment. The idea of this "double downsampling" stems from CT-ICP [10], the so far best performing open-source LiDAR odometry system on KITTI.

Most voxelization approaches, however, select the center of each occupied voxel to downsample the point cloud [25], [36]. Instead, we found it advantageous to maintain the original point coordinates, select only one point per voxel for a single scan, and keep its coordinates to avoid discretization errors. This means the reduced cloud is a subset of the deskewed one, i.e., $\hat{\mathcal{P}}^* \subseteq \mathcal{P}^*$. In our implementation, we keep only the first point that was inserted into the voxel.

C. Step 3: Local Map and Correspondence Estimation

In line with prior work [1], [10], [19], [35], we register the deskewed and subsampled scan $\hat{\mathcal{P}}^*$ to the point cloud built so far, i.e., a local map, to compute an incremental pose estimate $\Delta\mathsf{T}_{\text{icp}}$. We use frame-to-map registration as it proves more reliable and robust than the frame-to-frame alignment [1], [19]. To do that effectively, we must define a data structure representing the previously registered scans.

Modern approaches have used very different types of representations for this local map. Popular approaches are voxel grids [35], triangle meshes [31], surfel representations [1], or implicit representations [11]. As mentioned in Sec. III-B, we utilize a voxel grid to store a subset of 3D points. We use a grid with a voxel size of $v \times v \times v$ and store up to N_{max} points per voxel. After registration, we update the voxel grid by adding the points $\{\mathsf{T}_t\, \boldsymbol{p} \,|\, \boldsymbol{p} \in \mathcal{P}^*_{\text{merge}}\}$ from the new scan using the global pose estimate T_t. Voxels that already contain N_{max} points are not updated. Additionally, given the current pose estimate, we remove voxels outside the maximum range r_{max}. Thus, the size of the map will stay bounded.

Instead of a 3D array, we use a hash table to store the voxels, allowing a memory-efficient representation and fast nearest neighbor search [10], [20]. However, the used data structure can be easily replaced with VDBs [18], [32], Octrees [30], [34], or KD-Trees [2].

D. Adaptive Threshold for Data Association

ICP typically performs a nearest neighbor data association to find corresponding points between two point clouds [3]. When searching for associations, it is common to impose a maximum distance between corresponding points, often using a value of 1 m or 2 m [1], [31], [35]. This maximum distance threshold can be seen as an outlier rejection scheme, as all correspondences with a distance larger than this threshold are considered outliers and are ignored.

The required value for this threshold τ depends on the expected initial pose error, the number and type of dynamic objects in the scene, and, to some degree, the sensor noise. It is typically selected heuristically. Based on the considerations about the constant velocity motion prediction in Sec. III-A, we can, however, estimate a likely limit from data by analyzing how much the odometry may deviate from the motion prediction over time. This deviation $\Delta\mathsf{T}$ in the pose corresponds exactly to the local ICP correction to be applied to the predicted pose (but it is not known beforehand). Intuitively, we can observe the robot's acceleration in the magnitude of $\Delta\mathsf{T}$. If the robot is not accelerating, then $\Delta\mathsf{T}$ will have a small magnitude, often around zero, meaning that the constant velocity assumption holds and no correction has to be done by ICP.

We integrate this information into our data association search by exploiting the so-far successful ICP executions. We can estimate the possible point displacement between corresponding points in successive scans in the presence of a potential acceleration expressed through $\Delta\mathsf{T}$ as:

$$\delta(\Delta\mathsf{T}) = \delta_{\text{rot}}(\Delta\boldsymbol{R}) + \delta_{\text{trans}}(\Delta\boldsymbol{t}), \tag{5}$$

where $\Delta\boldsymbol{R} \in SO(3)$ and $\Delta\boldsymbol{t} \in \mathbb{R}^3$ refer to the rotational and translational component of the deviation, given by

$$\delta_{\text{rot}}(\Delta\boldsymbol{R}) = 2\; r_{\text{max}} \sin\left(\frac{1}{2} \underbrace{\arccos\left(\frac{\text{tr}(\Delta\boldsymbol{R}) - 1}{2}\right)}_{\theta}\right) \tag{6}$$

$$\delta_{\text{trans}}(\Delta\boldsymbol{t}) = \|\Delta\boldsymbol{t}\|_2. \tag{7}$$

The term $\delta_{\text{rot}}(\Delta\boldsymbol{R})$ represents the displacement that occurs for a range reading with maximum range r_{max} subject to the rotation $\Delta\boldsymbol{R}$, see also Fig. 2. Note that Eq. (5) constitutes an upper bound for the point displacement as

$$\|\Delta\boldsymbol{R}\,\boldsymbol{p} + \Delta\boldsymbol{t} - \boldsymbol{p}\|_2 \leq \delta_{\text{rot}}(\Delta\boldsymbol{R}) + \delta_{\text{trans}}(\Delta\boldsymbol{t}), \tag{8}$$

which follows from the triangle inequality.

For obtaining δ_{rot} other approaches could be considered, like taking into account the individual ranges for the adaptive threshold computation [4]. In our tests, we did not see any difference in the results but a 3-fold increase of the overall runtime; thus, we use $r_{\max}$ instead of r for computing δ_{rot}.

To compute the threshold τ_t at time t, we consider a Gaussian distribution over δ using the values of Eq. (5) over the trajectory computed so far whenever the deviation was larger than a minimum distance $\delta_{\min}$, i.e., situations where the robot's motion was deviating from the constant velocity model. Its standard deviation is

$$\sigma_t = \sqrt{\frac{1}{|\mathcal{M}_t|} \sum_{i \in \mathcal{M}_t} \delta(\Delta T_i)^2}, \tag{9}$$

where the index set $\mathcal{M}_t$ of deviations up to t is given by

$$\mathcal{M}_t = \{i \mid i < t \wedge \delta(\Delta T_i) > \delta_{\min}\}. \tag{10}$$

This avoids reducing the value of σ_t too much when the robot is not moving or is moving at constant velocity for a long time. In our experiments, we set this threshold $\delta_{\min}$ to 0.1 m. We then compute the threshold τ_t as the three-sigma bound $\tau_t = 3\,\sigma_t$, which we use in the next section for the data association search.

E. Step 4: Alignment Through Robust Optimization

We base our registration on classic point-to-point ICP [3]. The advantage of this choice is that we do not need to compute data-dependent features such as normals, curvature, or other descriptors, which may depend on the scanner or the environment. Furthermore, with noisy or sparse LiDAR scanners, features such as normals are often not very reliable. Thus, neglecting quantities such as normals in the alignment process is an explicit design decision that allows our system to generalize well to different sensor resolutions.

To obtain the global estimation of the pose T_t of the robot, we start by applying our prediction model $\mathsf{T}_{\text{pred},t}$ to the scan $\hat{\mathcal{P}}^*$ in the local frame. Successively, we transform it into the global coordinate frame using the previous pose estimate T_{t-1}, resulting in the source points

$$\mathcal{S} = \left\{ \boldsymbol{s}_i = \mathsf{T}_{t-1}\mathsf{T}_{\text{pred},t}\,\boldsymbol{p} \mid \boldsymbol{p} \in \hat{\mathcal{P}}^* \right\}. \tag{11}$$

For each iteration j of ICP, we obtain a set of correspondences between the point cloud $\mathcal{S}$ and the local map $\mathcal{Q} = \{\boldsymbol{q}_i \mid \boldsymbol{q}_i \in \mathbb{R}^3\}$ through nearest neighbor search over the voxel grid (Sec. III-C) considering only correspondences with a point-to-point distance below τ_t. To compute the current pose correction $\Delta\mathsf{T}_{\text{est},j}$, we perform a robust optimization minimizing the sum of point-to-point residuals

$$\Delta\mathsf{T}_{\text{est},j} = \underset{\mathsf{T}}{\operatorname{argmin}} \sum_{(s,q) \in \mathcal{C}(\tau_t)} \rho(\|\mathsf{T}\boldsymbol{s} - \boldsymbol{q}\|_2), \tag{12}$$

where $\mathcal{C}(\tau_t)$ is the set of nearest neighbor correspondences with a distance smaller than τ_t and ρ is the Geman-McClure robust kernel, i.e., an M-estimator with a strong outlier rejection property, given by

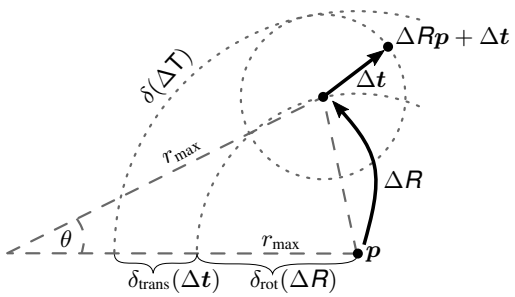


Fig. 2: Exemplary computation of the maximum point displacement $\delta(\Delta\mathsf{T})$ caused by a rotational and translational deviation $(\Delta R, \Delta\boldsymbol{t})$ from the predicted motion.

$$\rho(e) = \frac{e^2/2}{\underbrace{\sigma_t/3}_{\kappa_t} + e^2}, \tag{13}$$

where the scale parameter κ_t of the kernel is adapted online using σ_t. Lastly, we update the points $\boldsymbol{s}_i$, i.e.,

$$\{\boldsymbol{s}_i \leftarrow \Delta\mathsf{T}_{\text{est},j}\boldsymbol{s}_i \mid \boldsymbol{s}_i \in \mathcal{S}\}, \tag{14}$$

and repeat the process until the convergence criterion is met.

As a result of this process, we obtain the transformation $\mathsf{T}_t = \Delta\mathsf{T}_{\text{icp},t}\mathsf{T}_{t-1}\mathsf{T}_{\text{pred},t}$, where $\Delta\mathsf{T}_{\text{icp},t} = \prod_j \Delta\mathsf{T}_{\text{est},j}$. While we apply the prediction model $\mathsf{T}_{\text{pred},t}$ (i.e., the constant velocity prediction) to the local coordinate frame of the scan, we perform the ICP correction $\Delta\mathsf{T}_{\text{icp},t}$ in the global reference frame of the robot. This is done for efficiency reasons as it allows us to transform the source points $\mathcal{S}$ only once per ICP iteration. With this, the local pose deviation $\Delta\mathsf{T}_t$ at time t used in the Eq. (5) can be expressed as

$$\Delta\mathsf{T}_t = (\mathsf{T}_{t-1}\mathsf{T}_{\text{pred},t})^{-1}\,\Delta\mathsf{T}_{\text{icp},t}\mathsf{T}_{t-1}\mathsf{T}_{\text{pred},t}. \tag{15}$$

A standard termination criterion for the ICP algorithm is to control the number of iterations. Additionally, most approaches also have a further criterion based on the minimum change in the solution. Conversely, we found that controlling the number of iterations does not allow the algorithm to always find a good solution. Thus, we only employ the termination criterion based on the applied correction being smaller than γ, without imposing a maximum number of iterations.

Finally, the ICP correction is applied to the point cloud $\mathcal{P}^*_{\text{merge}}$, and the points are integrated into the local map.

F. Parameters

Our implementation depends on a small set of seven parameters. All are shown in Tab. I. We use the same parameters for all experiments. Most other approaches use a substantially larger set of parameters: MULLS [21] has 107 parameters, SuMa [1] has 49 parameters, and CT-ICP [10] has 30 parameters in their respective configuration files. In contrast, our approach only has two parameters for the correspondence search, four for the map representation and scan subsampling, and one for the ICP termination. Note that the maximum range of a scanner is a value that depends on the specific sensor in use and, as such, we do not consider it a system parameter.

Parameter	Value
Initial threshold τ_0	2 m
Min. deviation threshold δ_{min}	0.1 m
Max. points per voxel N_{max}	20
Voxel size map v	0.01 r_{max}
Factor voxel size map merge α	0.5
Factor voxel size registration β	1.5
ICP convergence criterion γ	10^{-4}

TABLE I: All seven parameters of our approach.

However, for some scenarios, the value of r_{max} might also be adapted to the specific environment in which the system is operating, e.g., not considering far away measurements that are usually less accurate.

IV. Experimental Evaluation

This work provides a simple yet effective LiDAR odometry pipeline that comes with a small set of parameters. We present our experiments to show the capabilities of our method. The results of our experiments support our key claims, namely that our approach (i) is on par with more complex state-of-the-art odometry systems, (ii) can accurately compute the robot's odometry in a large variety of environments and motion profiles with the same system configuration, and (iii) provides an effective solution to motion distortion without relying on IMUs or wheel odometers.

A. Experimental Setup

We use numerous datasets and common evaluation methods. We start with the KITTI odometry dataset [12] to evaluate our system against state-of-the-art approaches to LiDAR odometry. To investigate how we perform in other autonomous driving datasets employing a different sensor, we evaluate our approach on the MulRan dataset [15]. Additionally, we show that our approach can be used in different scenarios, such as the one present in the NCLT dataset [5], a segway dataset, and the Newer College dataset [22] recorded using a handheld device. We also analyze our method's different components, such as the motion-compensation scheme and the adaptive threshold.

Please note that due to space limitations we omit to show the results of the trajectories and a detailed runtime evaluation in this manuscript but refer the reader to the official project page where all the plots and per-sequence evaluation on the runtime performances are available.[1]

B. Performance on the KITTI-Odometry Benchmark

This experiment evaluates the performance of different odometry pipelines on the popular KITTI benchmark dataset. Since most systems do not do motion compensation, we use the already compensated KITTI scans for a fair comparison and disable the motion compensation for our approach and CT-ICP [10] in this first analysis (the performance of the motion compensation module will be studied later in Sec. IV-D1). Tab. II exhibits how our system challenges most state-of-the-art systems, which are typically more sophisticated than our point-to-point ICP. Based on the official KITTI Benchmark,

[1]https://www.github.com/PRBonn/kiss-icp/tree/main/evaluation

	Method	Seq. 00-10	Seq. 11-21
SLAM	SuMa++ [1]	0.70	1.06
	MULLS [21]	0.52	-
	CT-ICP [10]	0.53	**0.59**
Odometry	IMLS-SLAM [11]	0.55	0.69
	MULLS [21]	0.55	0.65
	F-LOAM [33]	0.84	1.87
	SuMa [1]	0.80	1.39
	Ours	**0.50**	0.61

TABLE II: KITTI Benchmark results with motion compensated data. We report the average relative translational error in % [13]. We compare across SLAM methods employing pose-graph optimization for improved results *(top)* and odometry methods *(bottom)*. We omit the relative rotational error, but these results are available at https://www.cvlibs.net/datasets/kitti/eval_odometry.php

we rank second among the open-source approaches (behind CT-ICP [10]) and ninth among all submissions. This indicates that our comparably simple system still performs better than all the publicly available systems out there, except CT-ICP [10]. Note that CT-ICP is a complete SLAM system, and it uses loop closures to correct for the accumulated drift of the odometry estimation. We in contrast obtain our results using only open-loop registration without any loop closing.

C. Comparison to State-of-the-Art Systems on Other Datasets

We proceed to analyze the performance of our system on different datasets, scenarios, and types of robots. For that, we use the MulRan dataset [15], a handheld device [22], and a segway dataset [5]. Odometry pipelines typically deal with those challenging scenarios but employ IMUs [27] or a different system configuration [10]. Our system performs on par with state-of-the-art systems using the same parameter values for all experiments and datasets. For this experiment, we compare against state-of-the-art odometry systems, namely MULLS [21], SuMa [1], F-LOAM [33], and CT-ICP [10]. Note that we do not provide an evaluation of CT-ICP for the MulRan dataset since CT-ICP does not provide support for this dataset.

For the MulRan dataset [15], we test the systems under evaluation on all available public sequences. Since the dataset provides three similar runs for each sequence, we report the average number of each sequence in Tab. III. Our method outperforms all state-of-the-art approaches by a large margin in both relative and absolute error.

We use both available sequences to evaluate the Newer College dataset and achieve similar results on the short experiment compared to CT-ICP. For the long experiment, the performance gap can be explained by the additional loop closing module of CT-ICP, which is a complete SLAM system. For the NCLT dataset experiment, we use the sequence evaluated on the original work of CT-ICP. We could not reproduce the results reported in CT-ICP [10] and therefore report the results given in the original paper [10] in Tab. IV. We achieve similar results than CT-ICP. However, we observed errors in the GPS ground-truth poses and missing frames. Therefore, the numbers on NCLT should be taken with a grain of salt and rather provide an estimate of how the systems perform. We discourage using NCLT to evaluate odometry systems:

Sequence	Method	Avg. tra.	Avg. rot.	ATE tra.	ATE rot.
KAIST	MULLS [21]	2.94	0.86	37.24	0.11
	SuMa [1]	5.59	1.73	43.61	0.14
	F-LOAM [33]	3.43	0.99	46.17	0.15
	Ours	**2.28**	**0.68**	**17.40**	**0.06**
DCC	MULLS [21]	2.96	0.98	38.35	0.12
	SuMa [1]	5.20	1.71	36.22	0.11
	F-LOAM [33]	3.83	1.14	42.70	0.13
	Ours	**2.34**	**0.64**	**15.16**	**0.05**
Riverside	MULLS [21]	5.42	2.21	91.16	0.16
	SuMa [1]	13.86	2.13	227.24	0.38
	F-LOAM [33]	5.47	1.18	138.09	0.22
	Ours	**2.89**	**0.64**	**49.02**	**0.08**
Sejong*	MULLS [21]	5.93	0.84	2151.00	0.49
	F-LOAM [33]	7.87	1.20	3448.97	0.82
	Ours	**4.69**	**0.70**	**1369.54**	**0.33**

TABLE III: Quantitative results on the MulRan dataset [15]. We report the relative translational error and the relative rotational error using the KITTI [13] metrics. Additionally, we show the absolute trajectory error for translation in m and for rotation in rad.

Method	NCD 01-short	NCD 02-long	NCLT 2012-01-8
MULLS [21]	0.82	1.23	-
F-LOAM [33]	2.02	fails	-
CT-ICP [10]	**0.48**	**0.58**	**1.17**
Ours	0.51	0.96	1.27

TABLE IV: Quantitative results for Newer College and NCLT. We report the relative translational error in % [13].

misalignments in the ground truth poses, missing frames, and inconsistencies in the data make the evaluation of odometry systems on such a dataset not a good evaluation tool from our perspective. However, we provide the results for completeness.

We show qualitative results in Fig. 1 generated using our KISS-ICP poses. Using a single system configuration, we can produce consistent maps on different sensor setups (Velodyne/Ouster vs. Livox) and different motion profiles (car, drone, segway, handheld) with the same parameters.

D. Ablation Studies

To understand how each component of our system impacts the odometry performance, we conduct ablation studies on the different components of our approach, namely, the motion compensation scheme and the adaptive threshold. To carry out these studies, we use the KITTI odometry dataset [12] as it is probably the best-known one.

1) Motion Compensation: To assess the impact of our motion compensation scheme, we utilize the raw LiDAR point clouds without any compensation applied. Note that the KITTI odometry benchmark point cloud data [12] is already compensated and, therefore, cannot be used for this study. Thus, we use the KITTI raw dataset [13]. We present the results in a familiar fashion, selecting only the sequences that correspond to the ones on the motion-compensated datasets [12]. As we can see in Tab. V, our motion compensation scheme can produce state-of-the-art results and is on par with substantially more sophisticated and thus complex compensation techniques such as the one introduced by CT-ICP [10]. Additionally, we study

Method	Avg. tra	Avg. rot	Avg. freq.
MULLS [21]	1.41	-	12 Hz
IMLS-SLAM [11]	0.71	-	1 Hz
CT-ICP [10]	0.55	-	15 Hz
Ours without deskewing	0.91	0.27	51 Hz
Ours + Deskewing (IMU)	0.51	0.19	38 Hz
Ours + Deskewing (CV)	**0.49**	0.16	38 Hz

TABLE V: Results of evaluating different state-of-the-art systems on KITTI-raw dataset (without motion compensation). We report the relative translational error and the relative rotational error using the KITTI [13] metrics. Additionally, we report the runtime operation of the systems being in consideration for this experiment.

Dataset	Data-Association Threshold τ				
	0.3 m	0.5 m	1.0 m	2.0 m	Ours
KITTI Seq. 00	0.54	**0.51**	0.53	0.55	**0.51**
KITTI Seq. 04	0.39	0.41	0.37	0.39	**0.36**
KITTI Avg. Seq. 00-10	0.53	0.51	0.51	0.53	**0.50**

TABLE VI: Comparison of different fixed thresholds vs. our proposed adaptive threshold on the KITTI dataset. We report the relative translational error in % [13].

how our system performs without applying motion compensation, as shown in Tab. V. We also evaluate the performance of our constant velocity model for motion compensation. To assess this, we compare the same compensation strategy but replace the velocity estimation with sensor data taken by the IMU. As seen in the results, our velocity estimation is on par or even slightly better with the IMU.

Besides the fact that CT-ICP's elastic formulation yields good results, our much simpler approach produces even better results. This result shows that the constant velocity model employed in our approach for compensating motion distortion is sufficient to cope with the slight reduction in performance when no compensation is applied. Consequently, we believe that more sophisticated techniques are unnecessary for *most* robotic odometry estimation.

2) Adaptive Data-Association Threshold: We finally evaluate how the adaptive threshold τ_t impacts the performance of our system by comparing it to a different set of fixed thresholds commonly used in open-source systems. To conduct this experiment, we identify the two KITTI sequences with the largest (00) and the smallest (04) average acceleration indicating different motion profiles. As we can see in Tab. VI, the best fixed threshold for sequence 00 is 0.5 m and 1.0 m for sequence 04. This means that a fixed threshold has to be tuned depending on the motion profile and thus to the dataset to achieve top performance. In contrast, our adaptive algorithm exploits the motion profile to estimate the threshold online, which results in on-par or better performance without the need to find a new fixed threshold for each sequence. Finally, our proposed adaptive threshold strategy achieves the best average result on the KITTI training sequences.

Please note that all the experiments from this ablation study use the robust kernel. For space reasons, we omitted the results of the evaluation of our system when no kernel is employed and only report the results averaged over the sequences. Not using the kernel produces 0.67% for the translational error and 0.25% for the rotational error [13].

V. Conclusion

This paper presents a simple yet highly effective approach to LiDAR odometry and shows that point-to-point ICP works very well – when used properly. Our approach operates solely on point clouds and does not require an IMU, even when dealing with high-frequency driving profiles. Our approach exploits the classical point-to-point ICP to build a generic odometry system that can be employed in different challenging environments, such as highway runs, handheld devices, segways, and drones. Moreover, the system can be used with different range-sensing technologies and scanning patterns. We only assume that point clouds are generated sequentially as the robot moves through the environment. We implemented and evaluated our approach on different datasets, provided comparisons to other existing techniques, supported all claims made in this paper, and released our code. The experiments suggest that our approach is on par with substantially more sophisticated state-of-the-art LiDAR odometry systems but relies only on a few parameters, and performs well on various datasets under different conditions with the same parameter set. Finally, our system operates faster than the sensor frame rate in all presented datasets. We believe this work will be a new baseline for future sensor odometry systems and a solid, high-performance starting point for future approaches. Our open-source code is robust and simple, easy to extend, and performs well, pushing the state-of-the-art LiDAR odometry to its limits and challenging most sophisticated systems.

VI. Acknowledgements

We thank Pierre Dellenbach for making his CT-ICP code available, which inspired our implementation. We thank Yue Pan for helping with the evaluation of MULLS for this paper. Thanks also to Igor Bogoslavskyi for his feedback.

References

[1] J. Behley and C. Stachniss. Efficient Surfel-Based SLAM using 3D Laser Range Data in Urban Environments. In *Proc. of Robotics: Science and Systems (RSS)*, 2018.

[2] J. Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.

[3] P. Besl and N. McKay. A Method for Registration of 3D Shapes. *IEEE Trans. on Pattern Analalysis and Machine Intelligence (TPAMI)*, 14(2):239–256, 1992.

[4] J.L. Blanco-Claraco. Mobile robot programming toolkit (mrpt). *URL: http://www. mrpt. org/*, 2014.

[5] N. Carlevaris-Bianco, A. Ushani, and R. Eustice. University of Michigan North Campus long-term vision and lidar dataset. *Intl. Journal of Robotics Research (IJRR)*, 35(9):1023–1035, 2016.

[6] N. Chebrolu, T. Läbe, O. Vysotska, J. Behley, and C. Stachniss. Adaptive Robust Kernels for Non-Linear Least Squares Problems. *IEEE Robotics and Automation Letters (RA-L)*, 6:2240–2247, 2021.

[7] X. Chen, S. Li, B. Mersch, L. Wiesmann, J. Gall, J. Behley, and C. Stachniss. Moving Object Segmentation in 3D LiDAR Data: A Learning-based Approach Exploiting Sequential Data. *IEEE Robotics and Automation Letters (RA-L)*, 6:6529–6536, 2021.

[8] X. Chen, A. Milioto, E. Palazzolo, P. Giguère, J. Behley, and C. Stachniss. SuMa++: Efficient LiDAR-based Semantic SLAM. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, 2019.

[9] Y. Chen and G. Medioni. Object modeling by registration of multiple range images. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, 1991.

[10] P. Dellenbach, J. Deschaud, B. Jacquet, and F. Goulette. CT-ICP Real-Time Elastic LiDAR Odometry with Loop Closure. In *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, 2022.

[11] J. Deschaud. Imls-slam: scan-to-model matching based on 3d data. In *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, 2018.

[12] A. Geiger, P. Lenz, and R. Urtasun. Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite. In *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2012.

[13] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun. Vision meets Robotics: The KITTI Dataset. *Intl. Journal of Robotics Research (IJRR)*, 32(11), 2013.

[14] T. Guadagnino, X. Chen, M. Sodano, J. Behley, G. Grisetti, and C. Stachniss. Fast Sparse LiDAR Odometry Using Self-Supervised Feature Selection on Intensity Images. *IEEE Robotics and Automation Letters (RA-L)*, 7(3):7597–7604, 2022.

[15] J. Jeong, Y. Cho, Y. Shin, H. Roh, and A. Kim. Complex urban lidar data set. In *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, 2018.

[16] R. Kolluri. Provably good moving least squares. *ACM Transactions on Algorithms (TALG)*, 4(2):1–25, 2008.

[17] J. Lin and F. Zhang. Loam_livox A Robust LiDAR Odemetry and Mapping LOAM Package for Livox LiDAR. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, 2019.

[18] K. Museth, J. Lait, J. Johanson, J. Budsberg, R. Henderson, M. Alden, P. Cucka, D. Hill, and A. Pearce. Openvdb: an open-source data structure and toolkit for high-resolution volumes. In *ACM SIGGRAPH Courses*. 2013.

[19] R.A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A.J. Davison, P. Kohli, J. Shotton, S. Hodges, and A. Fitzgibbon. KinectFusion: Real-Time Dense Surface Mapping and Tracking. In *Proc. of the Intl. Symposium on Mixed and Augmented Reality (ISMAR)*, 2011.

[20] M. Nießner, M. Zollhöfer, S. Izadi, and M. Stamminger. Real-time 3D Reconstruction at Scale using Voxel Hashing. In *Proc. of the SIGGRAPH Asia*, 2013.

[21] Y. Pan, P. Xiao, Y. He, Z. Shao, and Z. Li. MULLS: Versatile LiDAR SLAM Via Multi-Metric Linear Least Square. In *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, 2021.

[22] M. Ramezani, Y. Wang, M. Camurri, D. Wisth, M. Mattamala, and M. Fallon. The newer college dataset: Handheld lidar, inertial and vision with ground truth. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, 2020.

[23] V. Reijgwart, A. Millane, H. Oleynikova, R. Siegwart, C. Cadena, and J. Nieto. Voxgraph: Globally consistent, volumetric mapping using signed distance function submaps. *IEEE Robotics and Automation Letters (RA-L)*, 5(1):227–234, 2019.

[24] S. Rusinkiewicz and M. Levoy. Efficient variants of the ICP algorithm. In *Proc. of Int. Conf. on 3-D Digital Imaging and Modeling*, 2001.

[25] R.B. Rusu and S. Cousins. 3d is here: Point cloud library (pcl). In *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, 2011.

[26] J. Serafin and G. Grisetti. NICP: Dense Normal Based Point Cloud Registration. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, pages 742–749, 2015.

[27] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus. LIO-SAM Tightly-Coupled Lidar Inertial Odometry Via Smoothing and Mapping. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, 2020.

[28] T. Shan and B. Englot. LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, 2018.

[29] S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. MIT Press, 2005.

[30] E. Vespa, N. Nikolov, M. Grimm, L. Nardi, P. Kelly, and S. Leutenegger. Efficient octree-based volumetric slam supporting signed-distance and occupancy mapping. *IEEE Robotics and Automation Letters (RA-L)*, 3(2):1144–1151, 2018.

[31] I. Vizzo, X. Chen, N. Chebrolu, J. Behley, and C. Stachniss. Poisson Surface Reconstruction for LiDAR Odometry and Mapping. In *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, 2021.

[32] I. Vizzo, T. Guadagnino, J. Behley, and C. Stachniss. VDBFusion: Flexible and Efficient TSDF Integration of Range Sensor Data. *Sensors*, 22(3):1296, 2022.

[33] H. Wang, C. Wang, C. Chen, and L. Xie. F-LOAM: Fast LiDAR Odometry and Mapping. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, 2021.

[34] M. Zeng, F. Zhao, J. Zheng, and X. Liu. Octree-based fusion for realtime 3d reconstruction. *Graphical Models*, 75(3):126–136, 2013.

[35] J. Zhang and S. Singh. LOAM: Lidar Odometry and Mapping in Real-time. In *Proc. of Robotics: Science and Systems (RSS)*, 2014.

[36] Q. Zhou, J. Park, and V. Koltun. Open3D: A modern library for 3D data processing. *arXiv:1801.09847*, 2018.